Instituto Politécnico Nacional (IPN)

Escuela Superior de Cómputo (ESCOM)

Carrera:

Ingeniería en Inteligencia Artificial

Unidad de aprendizaje:

Algoritmos Bioinspirados

Practica 2:

Programacion Genetica

Maestra:

Abril Valeria Uriarte Arcia

Alumno:

Carranza Mercado Jesús Eduardo

20 / 03 / 26

Modifcaciones implementadas al codigo.

Modificación 1: Adaptación de la dimensionalidad de entrada.

El algoritmo original estaba diseñado para resolver problemas unidimensionales, recibiendo un único parámetro de entrada. Dado que la nueva función objetivo depende de dos variables independientes, se modificó la dimensionalidad del conjunto de primitivas (PrimitiveSet) de 1 a 2. Asimismo, se renombraron los argumentos a 'x' e 'y' para preservar la coherencia matemática y facilitar la legibilidad del árbol de sintaxis resultante.

```
pset = gp.PrimitiveSet("MAIN", 2)
```

```
pset.renameArguments(ARG0='x', ARG1='y')
```

Modificación 2. Inclusión de Constantes Efímeras.

Analizando la estructura de la función objetivo, se identificó la presencia de escalares específicos (el coeficiente 5 y el divisor 2). El código base carecía de generación de constantes, lo que obligaría al algoritmo a deducir estos valores mediante operaciones redundantes (fenómeno de bloat). Para mitigar esto y optimizar la convergencia, se introdujeron Constantes Aleatorias Efímeras (ERCs) con una distribución uniforme entre 1 y 5, permitiendo al modelo integrar escalares de forma nativa en sus genes.

```
pset.addEphemeralConstant("rand", lambda: random.randint(1, 5))
```

Modificación 3. Redefinición de la Función Objetivo y Entorno de Evaluación.

El mecanismo de evaluación original medía el Error Cuadrático Medio sobre una línea de puntos 1D para un polinomio distinto. Se rediseñó la función evalSymbReg para que evalúe pares ordenados (x, y) y calcule el error frente a la nueva superficie matemática requerida. Consecuentemente, el espacio de entrenamiento se actualizó de un vector unidimensional a una cuadrícula (grid) bidimensional, asegurando que los individuos sean probados en combinaciones cruzadas de ambas variables.

```
def evalSymbReg(individual, points):
    func = toolbox.compile(expr=individual)
    # Se evalúa la diferencia cuadrática entre el árbol y la función objetivo 3D
    sqerrors = ((func(x, y) - ((x**3) * 5 * (y**2) + x/2))**2 for x, y in points)
    return math.fsum(sqerrors) / len(points),

# Se genera una cuadrícula 2D de puntos (x, y) para evaluar la superficie
puntos_evaluacion = [(x/10., y/10.) for x in range(-10, 10) for y in range(-10, 10)]
toolbox.register("evaluate", evalSymbReg, points=puntos_evaluacion)
```

Modificación 4. Ajuste de Hiperparametros Evolutivos.

Al incrementar la dimensionalidad y agregar constantes, el espacio de búsqueda se expandió exponencialmente. Para evitar estancamientos en mínimos locales, se

incrementó el tamaño de la población de 300 a 1000 individuos y el número de generaciones de 40 a 100. Además, se elevó la probabilidad de cruza (cxpb) al 80% y la de mutación (mutpb) se dejo en un valor del 10%, tambien se aumentaron las generaciones, llegando hasta 200, promoviendo una mayor diversidad genética y una exploración más profunda del nuevo espacio de soluciones. Este fue sin duda el paso mas complicado del programa, ya que se necesito hacer prueba y error a lo largo de bastante tiempo para dar con la configuracion ideal para abordar este problema.

```
pop = toolbox.population(n=1000)
```

```
pop, log = algorithms.eaSimple(pop, toolbox, cxpb=0.8, mutpb=0.1, ngen=200,
                               stats=mstats, halloffame=hof, verbose=True)
```

Modificación 5. Depuración del Conjunto de Primitivas para Mitigación de Bloat.

En las primeras ejecuciones, se observó que el algoritmo convergía rápidamente a un error bajo, pero generaba individuos desproporcionadamente grandes e incomprensibles (fenómeno de bloat). El análisis de la ecuación resultante reveló un uso excesivo de funciones trigonométricas (seno y coseno) para aproximar la superficie. Dado que la función objetivo ($x^3 * 5y^2 + x/2$) es de naturaleza polinomial y algebraica, se eliminaron los operadores trigonométricos del conjunto de primitivas. Esto restringió el espacio de búsqueda, obligando al algoritmo a encontrar una solución basada puramente en aritmética básica, reduciendo drásticamente la complejidad del árbol y mejorando la interpretabilidad del modelo final.

```
# pset.addPrimitive(math.cos, 1)
# pset.addPrimitive(math.sin, 1)
```

Modificación 6. Implementacion de Primitivas Personalizadas (Atajos y Límites).

Se introdujeron primitivas personalizadas para el cálculo del cuadrado (sq) y el cubo (cube). Esto proporciona al algoritmo atajos semánticos que reducen la necesidad de ramificaciones profundas.

```
def sq(x): return x**2
def cube(x): return x**3
```

```
pset.addPrimitive(sq, 1)
pset.addPrimitive(cube, 1)
```

Modificación 7. Restricción de profundidad.

Se modificaron los decoradores de los operadores genéticos (mate y mutate) para reducir el límite de altura máxima del árbol sintáctico de 17 a 6. Esta fuerte presión selectiva hacia la parsimonia penaliza las estructuras redundantes y fuerza al modelo a priorizar la exactitud matemática con la menor cantidad de nodos posibles.

```
toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=6))
toolbox.decorate("mutate", gp.staticLimit(key=operator.attrgetter("height"), max_value=6))
```

Modificación 8. Manejo de Inestabilidad Numérica (Desbordamiento Aritmético).

Tras la inclusión de primitivas de potenciación (sq y cube), se observó la aparición de individuos numéricamente inestables. Las mutaciones aleatorias generaban árboles sintácticos con encadenamientos profundos de potencias (ej. x^{27}), provocando un desbordamiento aritmético (OverflowError) al rebasar el límite de precisión de coma flotante durante la evaluación del error cuadrático. Para mitigar esta vulnerabilidad, se introdujo un manejador de excepciones (try-except) en la función de evaluación (evalSymbReg). Ahora, cualquier individuo que resulte en un desbordamiento es penalizado con una aptitud infinita (float('inf')), erradicando inmediatamente esa rama genética inestable de la población sin interrumpir el flujo del algoritmo.

```
def evalSymbReg(individual, points):
    func = toolbox.compile(expr=individual)
    try:
        # Usamos corchetes para evaluar todos los puntos aquí mismo
        sqerrors = [(func(x, y) - ((x**3) * 5 * (y**2) + x/2))**2 for x, y in points]
        return math.fsum(sqerrors) / len(points),
    except OverflowError:
        # Si la ecuación explota, le damos el peor puntaje posible (infinito)
        return float('inf'),
```

Modificación 9. Supresión de Advertencias de Ejecución (RuntimeWarnings).

Al evaluar funciones con alto grado de inestabilidad, la biblioteca NumPy emite advertencias en la salida estándar (stderr) al encontrar valores nulos (nan) o infinitos (inf). Dado que el algoritmo ya maneja estos casos castigando la aptitud del individuo mediante la presión selectiva, se configuró el entorno (numpy.seterr) para silenciar estas advertencias. Esto mantiene la legibilidad en la bitácora de evolución sin alterar la lógica de penalización del algoritmo.

```
numpy.seterr(all='ignore')
```

Modificación 10. Post-procesamiento de Intérprete y Simplificación Simbólica.

Como resultado inherente del proceso estocástico, la Programación Genética tiende a generar árboles de sintaxis con alta redundancia algebraica (intrones estructurales), como multiplicaciones por la unidad derivadas de la división de una variable por sí misma (y/y). Para obtener un modelo matemáticamente interpretable y reducir el costo computacional de evaluación, se implementó una etapa de post-procesamiento utilizando la biblioteca de álgebra computacional SymPy. Mediante el mapeo de las primitivas de DEAP a operadores

algebraicos estándar, fue posible traducir el genotipo crudo, expandir los polinomios anidados y cancelar algorítmicamente los términos nulos y redundantes, revelando la verdadera ley matemática subyacente que el modelo evolutivo descubrió.

```
6    import sympy
```

```python
print("\n--- Post-procesamiento: Simplificación Algebraica ---")
# 1. Declaramos nuestras variables matemáticas reales
x_sym, y_sym = sympy.symbols('x y')

# 2. Diccionario para traducir el lenguaje de DEAP a matemáticas de SymPy
mapeo_operaciones = {
    'add': lambda a, b: a + b,
    'sub': lambda a, b: a - b,
    'mul': lambda a, b: a * b,
    # Para la simplificación, tratamos la división protegida como una división normal
    'protectedDiv': lambda a, b: a / b,
    'neg': lambda a: -a,
    'sq': lambda a: a**2,
    'cube': lambda a: a**3,
    'x': x_sym,
    'y': y_sym
}

try:
    # 3. Convertimos el texto del mejor individuo en una ecuación ejecutable
    ecuacion_cruda = eval(str(hof[0]), {"__builtins__": None}, mapeo_operaciones)

    # 4. Magia de SymPy: Expandimos y simplificamos los términos
    ecuacion_limpia = sympy.expand(ecuacion_cruda)

    print("Ecuación Original (Cruda):", str(hof[0]))
    print("\nEcuación Simplificada (SymPy):")
    print(ecuacion_limpia)

except Exception as e:
    print("No se pudo simplificar matemáticamente:", e)
```

Conclusion.

Para concluir esta práctica, puedo decir que el desarrollo de este algoritmo fue un verdadero reto de ingeniería, donde apliqué los conceptos teóricos de Programación Genética vistos en clase directamente sobre un problema de optimización multidimensional. Gran parte de mi trabajo se basó en la intuición matemática y en un ciclo exhaustivo de prueba y error; estuve modificando, evaluando y volviendo a ajustar los valores de los hiperparámetros repetidas veces hasta dar con la configuración ideal.

A lo largo de la práctica, seguí una serie de pasos críticos para guiar la evolución del modelo. Inicié adaptando el entorno para procesar dos variables independientes y redefiniendo la función objetivo. Al observar que el algoritmo generaba ecuaciones gigantescas y redundantes (bloat), tuve que intervenir restringiendo el espacio de búsqueda: eliminé las funciones trigonométricas, introduje primitivas personalizadas para potencias, e integré constantes efímeras enteras.

A pesar de estos ajustes, me enfrenté a diversos obstáculos, como desbordamientos aritméticos y estancamientos prematuros en mínimos locales. Logré superar estos problemas aplicando un manejo estricto de excepciones y calibrando cuidadosamente la presión selectiva, limitando la altura del árbol a 6 e incrementando la población y las generaciones para garantizar una correcta diversidad genética.

Finalmente, la integración de SymPy como herramienta de post-procesamiento fue clave para auditar el resultado crudo del mejor individuo. Tras múltiples iteraciones, logré que el algoritmo convergiera en una solución excepcionalmente precisa, demostrando empíricamente que la computación evolutiva no requiere de suerte, sino de una sintonización paciente y un profundo análisis crítico para acotar el problema y guiar la aleatoriedad hacia una solución matemática elegante.

```
190     798     357590          190     2.44142e+08     0.00574621      798     8.13256e+06     18.528  190     38      1       798     4.90033
191     818     96.4787         191     86655.1         0.00574621      818     2748.63         18.42   191     38      1       818     5.22624
192     830     1e+20           192     1e+23           0.00574621      830     3.1607e+21      18.72   192     35      1       830     4.57226
193     809     31052.9         193     1.67769e+07     0.00574621      809     679591          18.543  193     32      1       809     4.60675
194     857     7.41824e+15     194     7.41824e+18     3.60534e-33     857     2.34468e+17     18.416  194     37      1       857     5.0625
195     835     2.30209e+07     195     2.30209e+10     3.60534e-33     835     7.2762e+08      18.716  195     36      1       835     4.67476
196     819     284372          196     2.66054e+08     3.60534e-33     819     8.42839e+06     18.971  196     36      1       819     4.91184
197     833     26.1855         197     15704.3         0.00574621      833     507.837         18.686  197     37      1       833     4.92091
198     832     3.81483e+09     198     3.8147e+12      3.60534e-33     832     1.20571e+11     18.828  198     38      1       832     5.00324
199     828     43.0598         199     22893.2         3.60534e-33     828     867.78          18.687  199     36      1       828     5.00949
200     832     1.02461e+06     200     1.00099e+09     3.60534e-33     832     3.16413e+07     18.972  200     37      3       832     4.89829

¡Evolución terminada!
Mejor individuo encontrado (Ecuación):
add(mul(x, protectedDiv(1, 2)), mul(add(mul(sq(x), sq(y)), sq(mul(x, mul(y, 2)))), x))

--- Post-procesamiento: Simplificación Algebraica ---
Ecuación Original (Cruda): add(mul(x, protectedDiv(1, 2)), mul(add(mul(sq(x), sq(y)), sq(mul(x, mul(y, 2)))), x))

Ecuación Simplificada (SymPy):
5*x**3*y**2 + 0.5*x
jesuscarranza@192 Bioinspired-Algorithms %
```